

Student

I've seen `.pivot_table()` and `.melt()` in data engineering code but I never know when to use which. And time series operations like `.resample()` and `.rolling()` feel like magic to me.

ChatGPT • AI Assistant

CONCEPT CARD

This section solves two related problems. First, **reshaping**: data comes in two formats — **long** (one row per observation, preferred for analysis) and **wide** (one row per entity with many columns, preferred for reports). You need to move fluently between them. `pivot_table` converts long → wide; `melt` converts wide → long. Second, **time series**: once your data has a `DatetimeIndex`, Pandas unlocks a powerful toolkit — resampling to different time frequencies, rolling windows for smoothing, lag features for forecasting, and date accessor shortcuts.

We will use the same retail dataset from Section 2, extended with daily sales data for the time series subsections.

EXAMPLE CARD

Dataset setup for this section:

python — Code Example

```
import pandas as pd
import numpy as np

# Retail orders (from Section 2)
orders = pd.DataFrame({
    'order_id':    [1001,1002,1003,1004,1005,1006,1007,1008],
    'customer_id': [201, 202, 201, 203, 202, 204, 203, 201],
    'category':    ['Electronics','Clothing','Books','Electronics',
                    'Clothing','Clothing','Books','Electronics'],
    'region':      ['North','South','East','North','South','West','East','North'],
    'quantity':    [2, 1, 3, 1, 2, 4, 1, 2],
    'unit_price':  [500, 80, 25, 500, 80, 80, 25, 600],
    'order_date':  pd.to_datetime(['2024-01-05','2024-01-07','2024-01-07',
```

```

    '2024-01-10', '2024-01-12', '2024-01-
15',
    '2024-01-18', '2024-01-20'])

}))

orders['revenue'] = orders['quantity'] * orders['unit_price']

# Extended daily sales for time series – 90 days of data
np.random.seed(42)
dates = pd.date_range('2024-01-01', periods=90, freq='D')
daily_sales = pd.DataFrame({
    'date':    dates,
    'revenue': np.random.randint(800, 2000, 90) + np.arange(90) * 5,
    # upward trend
    'orders':  np.random.randint(10, 50, 90),
    'returns': np.random.randint(0, 8, 90)
})

daily_sales = daily_sales.set_index('date') # DatetimeIndex
print(daily_sales.head())

```

Student Learning Roadmap

RESHAPE

WIDE FORMAT — one row per entity, many columns (e.g. revenue_Jan, revenue_Feb)

Product	revenue_Jan	revenue_Feb	revenue_Mar
A	100	100	100
B	120	120	140
C	150	150	250

melt() / stack()

↑ ↓

pivot_table() / unstack()

LONG FORMAT — one row per observation (e.g. each product-month combination)

product	month	revenue
A	Jan	100
A	Feb	120
B	Jan	150
C	Jan	170
C	Jan	190

TIME SERIES

DatetimeIndex — set_index on date column

resample(freq) — change time granularity (daily → weekly)

rolling(window) — sliding window calculations

shift / diff / pct_change — lag and change features

orange = reshape operations

blue = time series operations

Section 3 Concept Map: Reshape and Time Series Operations

Wide vs Long Format: When to Use Each		
Aspect	Wide Format	Long Format
Structure	One row per entity; metrics as columns	One row per entity+metric combination

Aspect	Wide Format	Long Format
Example	product jan_rev feb_rev mar_rev	product month revenue
Preferred for	Reports, Excel, display, pivot tables	Analysis, GroupBy, plotting, ML
Storage efficiency	Fewer rows, many sparse columns	More rows, fewer denser columns
New metric addition	Add a new column (schema change)	Add new rows (no schema change)
Pandas function	pivot_table() / unstack()	melt() / stack()
SQL equivalent	PIVOT / CASE WHEN col expression	UNPIVOT / UNION ALL pattern

PIVOT_TABLE: LONG TO WIDE WITH AGGREGATION

ChatGPT • AI Assistant

A diagram showing a long-format source table on the left (columns: category, region, revenue — 8 rows of data) transforming via a pivot_table call into a wide-format grid on the right. The transformation arrow in the middle is labeled 'pd.pivot_table(values=revenue, index=category, columns=region, aggfunc=sum, fill_value=0)'. The output grid on the right shows: rows labeled with category values (Books, Clothing, Electronics), columns labeled with region values (East, North, South, West), and cell values showing aggregated revenue sums. One cell is highlighted in yellow labeled 'Books x North = 0 (fill_value replaced NaN)'. The margins=True effect is shown as an extra 'Total' row at the bottom and 'Total' column on the right, shaded grey. Parameter annotations point to the relevant parts of the output: 'index → rows', 'columns → col headers', 'values+aggfunc → cell values', 'margins → shaded Total'. Purpose: visually map each pivot_table parameter to its effect on the output layout.

pivot_table Parameters Mapped to Output Layout

Student

What exactly does `pivot_table` do? I've used Excel pivot tables but I'm not sure how the Pandas version maps to that.

CONCEPT CARD

`pd.pivot_table()` takes a **long-format** DataFrame and reshapes it into a **wide-format** summary table — exactly like an Excel PivotTable. You specify:

- ``values`` — the column to aggregate (e.g., `'revenue'``)
- ``index`` — what becomes the row labels
- ``columns`` — what becomes the column headers
- ``aggfunc`` — how to aggregate (default is `'mean'`` ; also `'sum'`` , `'count'`` , `np.sum` , etc.)
- ``margins=True`` — adds row/column subtotals (the 'Grand Total' row)
- ``fill_value=0`` — replace NaN with 0 when a combination has no data

🗨️ ANALOGY

Think of `pivot_table` as a summary spreadsheet generator. You tell it: 'put categories down the rows, regions across the columns, and put total revenue in each cell — and add a grand total row at the bottom'. Pandas does the rest.

EXAMPLE CARD

Example 1 — Basic: Revenue by category

python — Code Example

```
# Simple pivot — category as rows, sum of revenue
pivot1 = pd.pivot_table(
    orders,
    values='revenue',
    index='category',
    aggfunc='sum'
)
print(pivot1)
#               revenue
# category
# Books                100
# Clothing             560
# Electronics         2200
```

Example 2 — Intermediate: Category × Region cross-tab

python — Code Example

```
# Revenue by category (rows) × region (columns)
pivot2 = pd.pivot_table(
    orders,
    values='revenue',
    index='category',
    columns='region',
    aggfunc='sum',
    fill_value=0      # NaN → 0 where no orders for that combo
)
print(pivot2)
```

# region	East	North	South	West
# category				
# Books	75	0	0	0
# Clothing	0	0	160	320
# Electronics	0	2200	0	0

Student

What does `fill_value=0` do exactly? Why would there be NaN?

ChatGPT • AI Assistant

When a particular combination doesn't exist in the data — say no 'Books' were sold in the 'North' region — there is no row to aggregate. Pandas puts `NaN` there by default. `fill_value=0` replaces those NaN values with 0, which makes sense for revenue (no sales = 0 revenue) but might not make sense for averages or ratios.

Example 3 — Practical: Multiple aggfuncs + margins

python – Code Example

```
# Both sum and count, plus grand totals
pivot3 = pd.pivot_table(
    orders,
    values='revenue',
    index='category',
    columns='region',
    aggfunc=['sum', 'count'],
    fill_value=0,
    margins=True,      # Grand Total row and column
    margins_name='Total'
)
print(pivot3)
```

```
# Two-level columns: (sum, East), (sum, North), ..., (count, East), ...
# Bottom row 'Total' = column sums; rightmost col 'Total' = row sums
```

Student

I notice the result has multi-level columns when using multiple aggfuncs. How do I flatten those?

ChatGPT • AI Assistant

EXAMPLE CARD

python – Code Example

```
# Flatten multi-level column headers
pivot3.columns = ['_'.join(col).strip() for col in
pivot3.columns.values]
print(pivot3.columns.tolist())
# ['sum_East', 'sum_North', 'sum_South', 'sum_West', 'sum_Total',
#  'count_East', 'count_North', ... ]

# Or reset and rename manually if you want cleaner names
pivot3 = pivot3.reset_index()
```

Example 4 — Real Business: Monthly revenue per category

python – Code Example

```
# Add month column
orders['month'] = orders['order_date'].dt.strftime('%b') # 'Jan',
'Feb', etc.
orders['month_num'] = orders['order_date'].dt.month # For
sorting

monthly_cat = pd.pivot_table(
    orders,
    values='revenue',
    index='category',
    columns='month',
    aggfunc='sum',
    fill_value=0
)
print(monthly_cat)
```

```
# Since all our orders are in January:
# month      Jan
# category
# Books      100
# Clothing   560
# Electronics 2200
# (In real data with multiple months, each month becomes a column)
```

❗ COMMON PITFALL / MISTAKE

Incorrect — forgetting fill_value and operating on result:

python – Code Example

```
pivot = pd.pivot_table(orders, values='revenue', index='category',
                        columns='region', aggfunc='sum')
total = pivot.sum(axis=1) # NaN + number = NaN – wrong totals!
```

Correct — fill NaN before arithmetic:

python – Code Example

```
pivot = pd.pivot_table(orders, values='revenue', index='category',
                        columns='region', aggfunc='sum', fill_value=0)
total = pivot.sum(axis=1) # Now correctly sums 0s instead of NaN
```

🗂 INTERVIEW CARD

- **Frequency:** Moderate — common in data analyst and BI engineer roles
- **Question:** What is the difference between `pd.pivot_table()` and `df.groupby().agg()` ?
- **Expected Answer:** Both aggregate data, but `pivot_table` reshapes the result into a 2D wide-format table with group values as both rows and columns. `groupby().agg()` produces a long-format result with group keys as the index. `pivot_table` is essentially a `groupby` + `unstack` in one step.
- **Follow-up:** What does `margins=True` add to a pivot table?

❓ PRACTICE QUIZ

1. In `pd.pivot_table(df, values='rev', index='cat', columns='region', aggfunc='sum')`, what determines the number of columns in the result?
2. What is the default `aggfunc` in `pivot_table` ?
3. What does `fill_value=0` replace?

4. Predict: a `pivot_table` with `aggfunc=['sum', 'mean']` and 3 regions — how many columns in the output?
5. How do you add a 'Grand Total' row and column to a pivot table?
6. Debug: `pivot.sum(axis=1)` gives unexpected NaN — what is the likely cause?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 **ANSWER KEY**

1. The number of unique values in the `columns` parameter (unique regions)
2. `'mean'` — averages values in each cell
3. NaN values that appear when no data exists for a row/column combination
4. 6 columns — 2 `aggfuncs` × 3 regions
5. Set `margins=True` (and optionally `margins_name='Total'`)
6. NaN values in cells — fix by setting `fill_value=0` in the `pivot_table` call

❓ **CHECKPOINT QUESTION**

Before we move on — when would you choose `fill_value=0` vs leaving NaN in a pivot table? Give an example where filling with 0 would be misleading.

MELT AND STACK/UNSTACK: WIDE TO LONG AND BACK

ChatGPT • AI Assistant

A step-by-step transformation diagram. Starting state on the left labeled 'WIDE FORMAT (3 rows × 4 cols)': a table with columns 'category | Jan_revenue | Feb_revenue | Mar_revenue' and 3 data rows. A large rightward arrow labeled 'melt(id_vars=category, var_name=month, value_name=revenue)' points to the ending state on the right labeled 'LONG FORMAT (9 rows × 3 cols)': a table with columns 'category | month | revenue'. Color coding: the category column is highlighted green in both states (id_var — stays fixed). Each metric column (Jan, Feb, Mar) is a different shade of blue in the wide format, and each shade maps to a group of 3 rows in the long format showing the same color. An annotation reads: '3 rows × 3 metric cols = 9 rows'. Below the diagram, a second smaller diagram shows the inverse: `pivot_table(values=revenue, index=category, columns=month, aggfunc=sum)`. Purpose: make the melt transformation viscerally clear by color-coding which cells map where.

melt() Transformation: Wide to Long Format Step by Step

If `pivot_table` goes from long to wide, I'm guessing `melt` goes from wide to long? When would I actually need to do that?

ChatGPT • AI Assistant

CONCEPT CARD

`.melt()` is the inverse of pivot — it takes a wide-format DataFrame (many columns) and converts it to long format (one row per variable). This is one of the most important data cleaning operations because:

- Data from Excel reports, CSV exports, and BI tools is almost always wide-format
- Pandas GroupBy, Seaborn plotting, and most ML pipelines expect long format
- Long format is easier to extend — adding a new month means adding rows, not columns

Parameters:

- `'id_vars'` — columns to keep as-is (the identifier columns)
- `'value_vars'` — columns to unpivot (if omitted, all non-id columns are melted)
- `'var_name'` — name for the new 'variable' column (default: `'variable'`)
- `'value_name'` — name for the new 'value' column (default: `'value'`)

EXAMPLE CARD

Example 1 — Basic: Melt a simple wide report

python — Code Example

```
import pandas as pd

# Wide format — typical Excel report
wide = pd.DataFrame({
    'category': ['Electronics', 'Clothing', 'Books'],
    'Jan_revenue': [2200, 560, 100],
    'Feb_revenue': [1800, 720, 150],
    'Mar_revenue': [2500, 680, 200]
})
print(wide)
# category      | Jan_revenue | Feb_revenue | Mar_revenue
# Electronics   | 2200        | 1800        | 2500
# Clothing      | 560         | 720         | 680
# Books         | 100         | 150         | 200

# Melt to long format
```

```

long = wide.melt(
    id_vars = ['category'],
    value_vars = ['Jan_revenue', 'Feb_revenue', 'Mar_revenue'],
    var_name = 'month',
    value_name = 'revenue'
)
print(long)
# category      | month      | revenue
# Electronics   | Jan_revenue | 2200
# Electronics   | Feb_revenue | 1800
# Electronics   | Mar_revenue | 2500
# Clothing      | Jan_revenue | 560
# ... (9 rows total: 3 categories × 3 months)

```

Student

After melting, the 'month' column contains 'Jan_revenue' etc. — how do I clean that up?

ChatGPT • AI Assistant

EXAMPLE CARD**Example 2 — Intermediate: Clean up variable names after melt**

python — Code Example

```

# Strip '_revenue' from month names
long['month'] = long['month'].str.replace('_revenue', '', regex=False)
print(long['month'].unique()) # ['Jan', 'Feb', 'Mar']

# Full clean pipeline
long_clean = (
    wide.melt(id_vars='category',
              var_name='month', value_name='revenue')
    .assign(month = lambda df:
df['month'].str.replace('_revenue', '', regex=False))
    .sort_values(['category', 'month'])
    .reset_index(drop=True)
)
print(long_clean)

```

Example 3 — Practical: Melt multiple metric columns

python – Code Example

```
# Wide report with both revenue and order_count per month
wide_multi = pd.DataFrame({
    'category':      ['Electronics', 'Clothing', 'Books'],
    'Jan_revenue':    [2200, 560, 100],
    'Jan_orders':     [3, 3, 2],
    'Feb_revenue':    [1800, 720, 150],
    'Feb_orders':     [2, 4, 3]
})

# Melt only revenue columns first
rev_long = wide_multi.melt(
    id_vars = 'category',
    value_vars = ['Jan_revenue', 'Feb_revenue'],
    var_name = 'month_metric',
    value_name = 'revenue'
)
rev_long['month'] = rev_long['month_metric'].str[:3] # Extract 'Jan',
'Feb'
print(rev_long[['category', 'month', 'revenue']])
```

Example 4 — Real Business: Round-trip — pivot then melt

python – Code Example

```
# Start from our long orders data → pivot to wide report → melt back
to long
orders['month'] = orders['order_date'].dt.strftime('%b')

# Long → Wide (for a board report)
report = pd.pivot_table(orders, values='revenue',
                        index='category', columns='month',
                        aggfunc='sum', fill_value=0).reset_index()
print('Wide report:')
print(report)

# Wide → Long (for analysis pipeline)
back_to_long = report.melt(id_vars='category', var_name='month',
                          value_name='revenue')
print('Back to long:')
print(back_to_long.sort_values(['category', 'month']))
```

I've also seen `.stack()` and `.unstack()`. How are they different from `melt` and `pivot_table`?

ChatGPT • AI Assistant

`.stack()` and `.unstack()` work on the **index** rather than on regular columns:

- `.stack()` moves column labels into the row index (wide → long, index-based)
- `.unstack()` moves the innermost row index level into columns (long → wide, index-based)

They are particularly useful after a `groupby` that produces a MultiIndex:

python – Code Example

```
# groupby produces a Series with MultiIndex
grouped = orders.groupby(['category', 'region'])['revenue'].sum()
print(grouped)
# category      region
# Books         East      75
# Clothing      South    160
#               West     320
# Electronics   North   2200

# unstack: move 'region' from row index → columns
wide = grouped.unstack(level='region', fill_value=0)
print(wide)
# region      East  North  South  West
# category
# Books        75      0      0      0
# Clothing      0      0     160    320
# Electronics   0    2200      0      0

# stack: reverse – columns back to index
long_again = wide.stack()
print(long_again.head())
```

MEMORY TRICK CARD

- `.melt` / `.stack` → make it **taller** (more rows, fewer columns)
- `.pivot_table` / `.unstack` → make it **wider** (more columns, fewer rows)
- `.melt` operates on regular columns; `.stack` operates on the MultiIndex

🔗 INTERVIEW CARD

- **Frequency:** Moderate — asked in data engineering and analyst interviews
- **Question:** What is the difference between `.melt()` and `.stack()` in Pandas?
- **Expected Answer:** Both convert wide to long format, but `.melt()` operates on named column labels (regular DataFrame columns) and gives you control over the output column names via `var_name` and `value_name`. `.stack()` operates on the column index — it's index-based and works naturally after `groupby` operations that produce MultiIndex results.
- **Follow-up:** After `df.groupby(['A', 'B'])['val'].sum()`, what does `.unstack()` return?

🔗 PRACTICE QUIZ

1. How many rows does `wide.melt(id_vars='category')` produce if `wide` has 3 rows and 4 metric columns?
2. What parameter in `melt` names the column that holds the original column headers?
3. What is the inverse operation of `stack()` ?
4. Predict: `pd.DataFrame({'A':[1,2], 'B':[3,4], 'C':[5,6]}).melt(id_vars='A').shape`
5. In the round-trip `pivot_table → melt`, what information might be lost?
6. Debug: after `melt`, values in the 'month' column are 'Jan_revenue' instead of 'Jan' — how do you fix it?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔗 ANSWER KEY

1. 12 rows — 3 rows × 4 metric columns
2. `var_name` parameter
3. `unstack()`
4. (4, 3) — 2 rows × 2 non-id columns melted = 4 rows; columns are A, variable, value
5. Zero values from `fill_value=0` — after melting, you cannot distinguish original zeros from fill values unless you track them separately
6. `df['month'] = df['month'].str.replace('_revenue', '', regex=False)` or `.str[:3]`

? CHECKPOINT QUESTION

Given a DataFrame with columns `[product, Q1_sales, Q2_sales, Q3_sales, Q4_sales]` — write the `melt` call that produces columns `[product, quarter, sales]`.

TIME SERIES: DATETIMEINDEX, RESAMPLE, AND ROLLING

ChatGPT • AI Assistant

A two-panel diagram sharing a common input on the left: a vertical column of 14 small boxes representing 14 daily rows, each labeled with a day number (Day 1 through Day 14). Left panel labeled 'resample(W).sum()': the 14 daily boxes are grouped by dashed brackets — 7 boxes per group — collapsing into 2 output boxes labeled 'Week 1 total' and 'Week 2 total'. An annotation reads 'Output: 2 rows (from 14 daily rows) — granularity reduced'. Right panel labeled 'rolling(7).mean()': the 14 daily boxes remain as 14 output boxes, but the first 6 are shaded grey with a label 'NaN — window not yet full'. Boxes 7–14 are shaded blue with a sliding window bracket spanning 7 boxes shown sliding down the column. An annotation reads 'Output: 14 rows — same shape, NaN for incomplete windows'. A legend at bottom: resample = fewer rows + coarser grain; rolling = same rows + smoothed values. Purpose: visually clarify the fundamental difference between resample and rolling.

resample() vs rolling() — Output Shape and Use Case Comparison

Student

What makes a Pandas DataFrame a 'time series'? Is it just having a date column?

ChatGPT • AI Assistant

CONCEPT CARD

A DataFrame becomes a proper time series when its index is a `'DatetimeIndex'` — a special index type that Pandas understands as ordered time points. With a `DatetimeIndex`, you unlock three critical operations that don't work on regular DataFrames:

- `'resample(freq)'` — change time granularity (e.g., daily → weekly → monthly)
- `'rolling(window)'` — compute sliding window statistics (e.g., 7-day moving average)
- **Partial string indexing** — select rows with `df['2024-01']` to get all of January 2024

Without a `DatetimeIndex`, you can still work with dates, but these operations require manual `GroupBy` workarounds that are much more verbose.

EXAMPLE CARD

Example 1 — Basic: Creating a DatetimeIndex

python — Code Example

```
# Method 1: Set an existing date column as index
daily = daily_sales.copy() # already has DatetimeIndex from setup
print(type(daily.index))   # <class
                             'pandas.core.indexes.datetimes.DatetimeIndex'>
print(daily.index.freq)    # D (daily frequency)

# Method 2: Convert a string column then set as index
df = pd.DataFrame({'date': ['2024-01-01', '2024-01-02', '2024-01-03'],
                   'revenue': [1200, 1500, 900]})
df['date'] = pd.to_datetime(df['date'])
df = df.set_index('date')
print(df.index.dtype)      # datetime64[ns]

# Partial string indexing – select all of January 2024
jan = daily['2024-01']     # All January rows
print(f'January rows: {len(jan)}') # 31 rows

# Select a date range
q1 = daily['2024-01-01':'2024-03-31']
print(f'Q1 rows: {len(q1)}') # 90 rows (our full dataset)
```

Example 2 — Intermediate: resample() — change time frequency

python — Code Example

```
# Daily → Weekly (sum revenue, mean orders)
weekly = daily_sales.resample('W').agg({
    'revenue': 'sum',
    'orders': 'mean',
    'returns': 'sum'
})
print(weekly.head())

# date      | revenue | orders | returns
# 2024-01-07 | 8234    | 28.4   | 12
# 2024-01-14 | 9102    | 31.0   | 15
# ...

# Daily → Monthly
monthly = daily_sales.resample('ME').agg({'revenue': 'sum', 'orders':
    'sum'})
```

```
print(monthly)
# date          | revenue | orders
# 2024-01-31    | 47823   | 892
# 2024-02-29    | 54210   | 1023
# 2024-03-31    | 61540   | 1102
```

Student

What are the common frequency strings like 'W', 'ME'? I can never remember them.

ChatGPT • AI Assistant

The most common ones are covered in the reference table below. For `resample`, the label on the output represents the **end** of the period for 'ME' (month-end) and the **end of week** for 'W'. You can control the label with `label='left'` or `label='right'` and the closed interval with `closed='left'`.

Example 3 — Practical: `rolling()` — sliding window

python — Code Example

```
# 7-day rolling mean revenue (smoothes out day-of-week effects)
daily_sales['rev_7d_avg'] =
daily_sales['revenue'].rolling(window=7).mean()

# 30-day rolling mean
daily_sales['rev_30d_avg'] =
daily_sales['revenue'].rolling(window=30).mean()

print(daily_sales[['revenue', 'rev_7d_avg', 'rev_30d_avg']].head(35))
# First 6 rows: rev_7d_avg = NaN (not enough data for window of 7)
# Row 7 onward: rolling mean computed
# First 29 rows: rev_30d_avg = NaN
# Row 30 onward: 30-day mean computed
```

Student

Why is rolling mean useful? Why not just use `resample`?

ChatGPT • AI Assistant

They answer different questions:

- **`resample`** changes the frequency — you go from 90 daily rows to 3 monthly rows. The daily detail is gone.
- **`rolling`** keeps all your original rows but adds a smoothed view alongside each day. Perfect for charts that show both the noisy daily line and the smooth trend line together.

Rolling is also the foundation of feature engineering for time series forecasting — you compute rolling mean, rolling std, rolling max as input features for each day.

Example 4 — Real Business: Complete time series feature pipeline

python — Code Example

```
# Build a feature table for a forecasting model
features = daily_sales[['revenue']].copy()

# Rolling statistics
features['rev_7d_mean'] = features['revenue'].rolling(7).mean()
features['rev_7d_std']   = features['revenue'].rolling(7).std()
features['rev_30d_mean'] = features['revenue'].rolling(30).mean()

# Lag features
features['rev_lag1'] = features['revenue'].shift(1)    # Yesterday's revenue
features['rev_lag7'] = features['revenue'].shift(7)    # Same day last week

# Change features
features['rev_diff'] = features['revenue'].diff()      # Day-over-day change
features['rev_pct_change'] = features['revenue'].pct_change() # % change

# Date features
features['dayofweek'] = features.index.dayofweek      # 0=Mon, 6=Sun
features['is_weekend'] = features['dayofweek'].isin([5, 6]).astype(int)
features['month'] = features.index.month
features['is_month_end'] = features.index.is_month_end.astype(int)

# Drop rows with NaN from rolling/lag windows
features_clean = features.dropna()
print(f'Feature rows after dropna: {len(features_clean)}') # 90 - 30 =
```

60

```
print(features_clean.head(3))
```

❗ COMMON PITFALL / MISTAKE

Incorrect — calling resample without DatetimeIndex:

python — Code Example

```
daily_sales_reset = daily_sales.reset_index() # date is now a column,
not index
daily_sales_reset.resample('W') # TypeError: Only valid with
DatetimeIndex
```

Correct — ensure DatetimeIndex before resample:

python — Code Example

```
daily_sales_reset = daily_sales_reset.set_index('date') # Restore as
index
daily_sales_reset.resample('W').sum() # Now works
```

🔗 INTERVIEW CARD

- **Frequency:** High — time series is in every data engineering and ML engineering interview
- **Question:** What is the difference between `resample()` and `rolling()` in Pandas?
- **Expected Answer:** `resample` changes the time frequency of the entire DataFrame — e.g., daily to weekly — reducing the number of rows. `rolling` keeps the same number of rows but computes a window statistic alongside each row. Use `resample` when you want to aggregate to a coarser time grain; use `rolling` when you want a smoothed or lagged feature at the original frequency.
- **Follow-up:** Why do the first N-1 rows of a `.rolling(N).mean()` produce NaN?

❓ PRACTICE QUIZ

1. What must the DataFrame index be before calling `.resample()` ?
2. What does `daily.resample('W').sum()` produce — how many rows from 90 days?
3. What do the first 6 rows of `.rolling(7).mean()` contain and why?
4. Predict: `daily['revenue'].shift(1).iloc[0]` — what value?

5. What does `daily['2024-01']` return when `daily` has a DatetimeIndex?
6. Debug: `df.resample('ME').mean()` gives fewer rows than expected — what could cause this?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 **ANSWER KEY**

1. A DatetimeIndex (set via `set_index` on a datetime column)
2. Approximately 13 rows — 90 days / 7 days per week $\approx$ 13 weekly buckets
3. NaN — there are fewer than 7 data points before row 7, so the window is incomplete. Row 7 is the first row with a full 7-day window.
4. NaN — shifting down by 1 means the first row has no previous value
5. All rows where the index year is 2024 and month is January — partial string indexing
6. Gaps in the date index (missing dates) — `resample` creates buckets based on the date range, so sparse data still produces the right number of buckets. More likely cause: the DataFrame has a non-monotonic or non-datetime index — check `df.index.is_monotonic_increasing`

❓ **CHECKPOINT QUESTION**

Explain why a 30-day rolling mean feature table built on the full dataset (before train/test split) causes data leakage, and how you would fix it.

SHIFT, DIFF, PCT_CHANGE, AND THE DT ACCESSOR

ChatGPT • AI Assistant

A four-column table diagram with 7 data rows (Days 1–7). Column 1 labeled 'revenue' shows values: 1142, 1287, 973, 1521, 1103, 1456, 1678. Column 2 labeled 'shift(1) — lag' shows the same values shifted down one row: NaN, 1142, 1287, 973, 1521, 1103, 1456 — a grey NaN box at top, then each value from column 1 one row lower. Column 3 labeled 'diff(1) — absolute change' shows: NaN, +145, -314, +548, -418, +353, +222 — positive values in green, negative in red. Column 4 labeled 'pct_change(1) — relative change' shows: NaN, +12.7%, -24.4%, +56.3%, -27.5%, +32.0%, +15.2% — same color coding. Arrows along the left side indicate: 'shift = copy value from t-1'; 'diff = current minus shift'; 'pct_change = diff divided by shift'. A note at bottom: 'All three produce NaN for the first row(s) — account for this before modeling'. Purpose: make the mathematical relationship between the three operations visually explicit.

shift(), diff(), and pct_change() Illustrated on a 7-Day Revenue Series

I understand `rolling` now but I'm not sure when to use `.shift()`, `.diff()`, and `.pct_change()`. They all seem similar.

ChatGPT • AI Assistant

CONCEPT CARD

These three are all about **temporal relationships** — comparing a value to an earlier version of itself:

- `.shift(n)` — move values forward (positive n) or backward (negative n) by n periods. Creates a **lag feature** — the value at time t-n alongside time t.
- `.diff(n)` — computes `current - previous_n`. Equivalent to `series - series.shift(n)`. Shows **absolute change**.
- `.pct_change(n)` — computes `(current - previous_n) / previous_n`. Shows **relative (percentage) change**.

🗨️ ANALOGY

Think of `.shift(1)` as photocopying yesterday's newspaper and placing it next to today's. `.diff()` asks 'how much did the headline number change?'
`.pct_change()` asks 'by what percentage did it change?'

EXAMPLE CARD

Example 1 — Basic: shift, diff, pct_change side by side

python — Code Example

```
rev = daily_sales['revenue'].head(7)
print(pd.DataFrame({
    'revenue':    rev.values,
    'shift_1':    rev.shift(1).values,      # Yesterday's value
    'diff_1':     rev.diff(1).values,       # Today - Yesterday
    'pct_chg':    rev.pct_change(1).round(3).values # % change
})))
```

#	revenue	shift_1	diff_1	pct_chg
# 1142	1142	NaN	NaN	NaN
# 1287	1287	1142.0	145.0	0.127
# 973	973	1287.0	-314.0	-0.244
# 1521	1521	973.0	548.0	0.563
# 1103	1103	1521.0	-418.0	-0.275

```
# 1456      | 1103.0 | 353.0 | 0.320
# 1678      | 1456.0 | 222.0 | 0.152
```

Student

When would I use `.shift()` beyond just seeing yesterday's value?

ChatGPT • AI Assistant

Three primary use cases:

1. **Lag features for ML:** Add `revenue_yesterday` and `revenue_7days_ago` as input features to predict today's revenue
2. **Same-period comparison:** `revenue / revenue.shift(365) - 1` gives year-over-year growth
3. **Lead features:** `shift(-1)` (negative shift) gives you tomorrow's value — useful for creating a target variable in supervised learning

Example 2 — Intermediate: Building lag features

python – Code Example

```
lag_features = daily_sales[['revenue']].copy()
lag_features['lag_1d'] = lag_features['revenue'].shift(1)
lag_features['lag_7d'] = lag_features['revenue'].shift(7)
lag_features['lag_30d'] = lag_features['revenue'].shift(30)

# Year-over-year comparison (would need 365-day dataset)
# lag_features['yoy'] = lag_features['revenue'] /
lag_features['revenue'].shift(365) - 1

print(lag_features.head(10))
```

Example 3 — Practical: Period-over-period analysis

python – Code Example

```
# Monthly revenue with MoM (month-over-month) change
monthly_rev =
daily_sales['revenue'].resample('ME').sum().reset_index()
monthly_rev.columns = ['month', 'revenue']

monthly_rev['prev_month_rev'] = monthly_rev['revenue'].shift(1)
```

```
monthly_rev['mom_change'] = monthly_rev['revenue'].diff()
monthly_rev['mom_pct'] =
monthly_rev['revenue'].pct_change().mul(100).round(1)

print(monthly_rev)
# month      | revenue | prev_month | mom_change | mom_pct
# 2024-01-31 | 47823   | NaN        | NaN        | NaN
# 2024-02-29 | 54210   | 47823      | 6387       | 13.4%
# 2024-03-31 | 61540   | 54210      | 7330       | 13.5%
```

Student

How does the `dt` accessor work? I've seen `df['date'].dt.month` used a lot.

ChatGPT • AI Assistant

CONCEPT CARD

The **`dt` accessor** unlocks date component extraction on any datetime column (or `DatetimeIndex`). It mirrors Python's `datetime` attributes but works vectorized across the whole Series:

python — Code Example

```
# On a DatetimeIndex
print(daily_sales.index.month[:5])      # [1 1 1 1 1]
print(daily_sales.index.dayofweek[:5])  # [0 0 1 2 3] (0=Mon)
print(daily_sales.index.is_month_end)   # True only on last day of
month

# On a datetime column with .dt accessor
orders['order_date'].dt.year            # Year: 2024
orders['order_date'].dt.month           # Month: 1
orders['order_date'].dt.day             # Day: 5, 7, ...
orders['order_date'].dt.dayofweek       # 0=Monday ... 6=Sunday
orders['order_date'].dt.day_name()      # 'Friday', 'Sunday', ...
orders['order_date'].dt.quarter         # 1, 2, 3, or 4
orders['order_date'].dt.is_month_end    # True/False
orders['order_date'].dt.strftime('%Y-%m') # '2024-01'
```

Example 4 — Real Business: Date feature engineering for a reporting dashboard

python — Code Example

```
# Enrich orders with date features for slicing and dicing in a
dashboard
orders['year']          = orders['order_date'].dt.year
orders['month']         = orders['order_date'].dt.month
orders['month_name']    = orders['order_date'].dt.strftime('%b') #
'Jan'
orders['day_of_week']   = orders['order_date'].dt.dayofweek      # 0-6
orders['weekday']       = orders['order_date'].dt.day_name()     #
'Monday'
orders['is_weekend']    = orders['order_date'].dt.dayofweek.isin([5, 6])
orders['quarter']       = orders['order_date'].dt.quarter
orders['week_num']      = orders['order_date'].dt.isocalendar().week

print(orders[['order_date', 'month_name', 'weekday', 'is_weekend', 'quarter']].head())
```

❗ COMMON PITFALL / MISTAKE

Incorrect — using dt accessor on non-datetime column:

python – Code Example

```
df['date_str'] = '2024-01-15' # This is an object/string dtype
df['date_str'].dt.month # AttributeError: Can only use .dt accessor
with datetimelike values
```

Correct — convert to datetime first:

python – Code Example

```
df['date'] = pd.to_datetime(df['date_str'])
df['date'].dt.month # Works correctly
```

🌐 REAL WORLD APPLICATION

In e-commerce analytics, the most impactful date features are: `is_weekend`, `day_of_week`, `is_month_end` (payday effect), `week_num` (promotional cycles), and `is_holiday` (requires a custom calendar). These features explain a large portion of daily revenue variance and are the first things a data engineer adds when building a time series model.

🔮 AI DEEP DIVE PROMPT

Copy this into Claude or ChatGPT:

"Act as a senior data engineer. I want to master Pandas time series operations. Give me 10 progressively harder exercises using a daily sales dataset (date, revenue, orders, returns). Exercises should cover: DatetimeIndex creation, resample at different frequencies, rolling windows of varying sizes, shift and lag feature creation, pct_change for period comparisons, dt accessor feature engineering, and combining all of these into a feature table. For each exercise: state the business goal, ask me to write the code, after I attempt it show the optimal solution, explain any time series gotcha (NaN from rolling, data leakage, index alignment issues), and give one interview question. Do not continue until I answer."

🔥 EXERCISE

Using `daily_sales` (90 days, DatetimeIndex):

1. **Reshape exercise:** Resample *daily_sales* to weekly frequency, computing sum of revenue, mean of orders, and max of returns. Then melt the result into long format with columns `[weekend_date, metric, value]`.
2. **Feature engineering exercise:** Build a complete feature table with: 7-day and 30-day rolling mean revenue, 1-day and 7-day lag revenue, day-over-day *pctchange*, *dayofweek* (0–6), *isweekend* (0/1), and month. Drop NaN rows. Report the final shape.

SUMMARY CARD

Section 3 Key Takeaways:

- **Long format** is preferred for analysis; **wide format** for reporting and display
- `pivot_table(values, index, columns, aggfunc)` → long to wide with aggregation; `margins=True` adds subtotals; `fill_value=0` prevents NaN in calculations
- `melt(id_vars, var_name, value_name)` → wide to long; inverse of `pivot_table`
- `stack()` / `unstack()` → same as melt/pivot but index-based, used after `groupby`
- `set_index(datetime_col)` → creates DatetimeIndex; required before `resample` and `rolling`
- `resample(freq).agg(...)` → changes time granularity, reduces rows
- `rolling(n).mean()` → sliding window, same row count, NaN for first n-1 rows
- `shift(n)` → lag feature; `diff()` → absolute change; `pct_change()` → % change
- `.dt` accessor → extract date components from datetime columns or DatetimeIndex
- Always compute rolling/lag features AFTER train/test split to avoid data leakage

? PRACTICE QUIZ

1. What method converts a wide DataFrame to long format?
2. What does `resample('ME').sum()` do to a daily DataFrame with 90 rows?
3. Why does `.rolling(30).mean()` return NaN for the first 29 rows?
4. Predict: `pd.Series([10, 20, 15, 25]).pct_change()` — what are the values?
5. What attribute of DatetimeIndex returns `True` for the last day of each month?
6. Debug: `df['date'].dt.month` raises `AttributeError` — what is wrong?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. `.melt()`
2. Reduces to 3 rows (one per month: Jan, Feb, Mar) with revenue summed per month
3. The rolling window requires 30 data points to compute a mean — rows 0 through 28 have fewer than 30 preceding values
4. `[NaN, 1.0, -0.25, 0.667]` — NaN for first; $(20-10)/10=1.0$; $(15-20)/20=-0.25$; $(25-15)/15=0.667$
5. `.is_month_end`
6. The 'date' column has dtype `object` (string) — fix with `df['date'] = pd.to_datetime(df['date'])` first

? CHECKPOINT QUESTION

You have daily sales data for 2 years. You want to build a model to predict tomorrow's revenue. List four time series features you would create using the operations from this section, and explain why each feature might help the model.

🌟 CHAPTER REVIEW & SUMMARY**What You Learned:**

Understanding of the core concepts presented in this chapter including pivot, melt, reshape, and time series, their architectural components, practical use cases, and design paradigms.

Quick Revision Notes:

- Key definitions and nomenclature for the chapter topic.
- Hands-on code execution patterns and setup checklists.
- Analogy-driven visualization of data structures or layered setups.

Core Visualized Components:

- **Section 3 Concept Map: Reshape and Time Series Operations:** A two-column concept map. Left column labeled 'RESHAPE' contains two boxes conne...